ALUMNICONNECT / DBMS PROJECT DESIGN DOCUMENT

# Redis Design Specification

## Key naming • Data structures • TTL policies • Cache invalidation

A practical Redis architecture for the Alumni–Student Mentorship and Mock Interview Platform.

| DOCUMENT CONTROL | VALUE |
|---|---|
| Project | AlumniConnect - Alumni–Student Mentorship Platform |
| Document | Redis Design Specification |
| Version / date | 1.0 / 02 November 2026 |
| Backend baseline | FastAPI modular monolith |
| Authoritative database | MySQL 8.x with InnoDB |
| Supporting stores | Redis cache / temporary state; Neo4j recommendation projection |
| Status | Implementation-oriented design; configurable values marked as recommendations |

**CENTRAL DESIGN RULE**

Redis accelerates reads and supports short-lived state. It is disposable and rebuildable. MySQL remains authoritative for identities, profiles, requests, allocations, availability, bookings, sessions, feedback, reviews and audit history.

Prepared from the supplied AlumniConnect DBMS Project Blueprint, Product Requirements Document, Application Architecture, Workflow and State Diagrams, Business Rules Catalogue and Conceptual ER Model.

# Document map

| SECTION | WHAT IT DEFINES |
|---|---|
| 01. Executive summary and design boundary | Redis role, ownership and non-goals |
| 02. Workload inventory | What should and should not be stored in Redis |
| 03. Key naming convention | Environment, version, resource, identifiers and query dimensions |
| 04. Key catalogue - read caches | Mentor profiles, catalogues, recommendation payloads and rankings |
| 05. Key catalogue - temporary state | Challenges, counters, idempotency, feeds and version keys |
| 06. Data structure selection | Hash, String, Set, Sorted Set, List and Stream choices |
| 07. TTL policy | Recommended expiry values, rationale and safeguards |
| 08. Cache-aside read path | Hit/miss flow, safe population and stampede control |
| 09. Invalidation design | Targeted deletion, version namespaces and event mapping |
| 10. Event-to-cache matrix | Which committed business events invalidate which cache keys |
| 11. Transactional outbox integration | Commit ordering, retries and idempotent invalidation |
| 12. Temporary state patterns | Login challenges, rate limiting and idempotency keys |
| 13. Redis command cookbook | redis-cli inspection and data-structure examples |
| 14. FastAPI integration templates | Python cache-aside and post-commit invalidation examples |
| 15-19. Operations and verification | Reliability, privacy, monitoring, acceptance tests and implementation plan |
| 20. Final design summary | Recommended first-release decisions |
| Appendices | Key register, configuration decisions and source basis |

**HOW TO INTERPRET THIS DOCUMENT**

“Blueprint-defined” means directly supported by your supplied project documents. “Recommended default” means a concrete implementation value or pattern proposed here because the sources establish the need but do not set the exact number or syntax. Confirm tunable policies with the team before final deployment.

# 01 / Executive summary and design boundary

AlumniConnect uses a modular monolith with polyglot persistence. FastAPI is the single logical API boundary. MySQL 8.x with InnoDB owns durable business records and transactional decisions. Redis is a performance layer for cacheable reads and short-lived state. Neo4j provides relationship-based mentor discovery. This Redis specification defines only the Redis responsibility; it does not move business ownership out of MySQL.

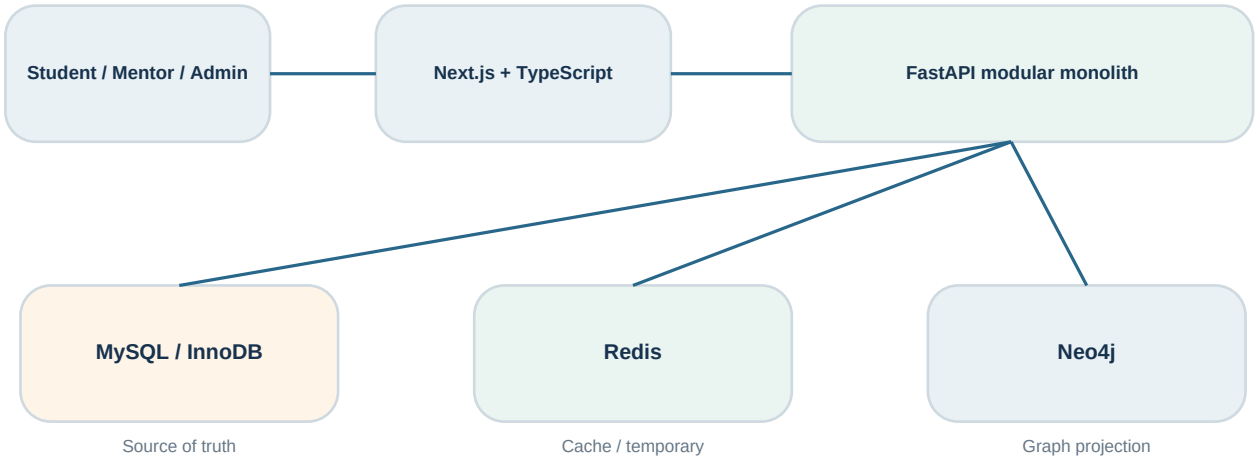



*Figure 1. Redis is accessed by the backend, not directly by the browser.*

## 1.1 Redis responsibilities

| RESPONSIBILITY | REDIS MAY DO | REDIS MUST NOT DO |
|---|---|---|
| Read acceleration | Cache public mentor cards, skill/domain catalogue data and recent recommendation responses. | Become the authoritative profile, skill or rating record. |
| Short-lived state | Store temporary login challenges, short-lived session support, rate-limit counters and idempotency records. | Store plaintext passwords or replace durable account/session controls. |
| Precomputed views | Maintain a temporary sorted ranking of popular mentors and a bounded recent-activity feed. | Replace SQL reports or historical activity records. |
| Availability display | Optionally cache a short-lived availability preview for display. | Confirm that a slot is free or approve a booking. |

## 1.2 Non-negotiable boundaries

- **MySQL wins on disagreement.** Redis entries are rebuildable and must not override authoritative rows.
- **Authorization always executes.** A cache hit is not permission to view a record.
- **Booking always revalidates in MySQL.** The booking transaction locks and validates the current availability row, allocation and overlap rules.
- **Outbox drives secondary changes.** A business update and its outbox event commit together; a worker invalidates Redis after commit.

**IMPORTANT**

A cached availability preview is a user-interface convenience only. It may become stale immediately after another user books the slot. The booking procedure must re-read and lock MySQL rows regardless of Redis contents.

# 02 / Workload inventory and cache boundaries

The data-structure choices follow the access pattern rather than the relational table layout. Cache display projections, not complete MySQL rows. Keep private fields out of public cache payloads. For temporary state, store only what the application needs to enforce a short-lived interaction.

| FEATURE | WHY REDIS HELPS | SOURCE-ALIGNED CHOICE |
|---|---|---|
| Frequently viewed mentor cards | Avoid repeatedly assembling the same public display projection. | HASH; cache-aside; invalidate after mentor profile changes. |
| Skills and domain catalogue | Reference data is read often and changes relatively infrequently. | HASH for ID-to-value lookup; optional SET for membership. Bounded TTL or explicit invalidation. |
| Recent recommendation results | Reuse an expensive graph + MySQL eligibility response for a short interval. | STRING containing JSON or HASH. Invalidate for relevant student/mentor/catalogue changes. |
| API rate limiting | Count attempts in a defined time window. | STRING counter with atomic increment and expiry. |
| Temporary login challenge | Hold a verification challenge only for its short validity period. | STRING with mandatory expiry. Never store a password. |
| Request idempotency | Recognise a retried API request and return the same result. | STRING with request fingerprint and processing/result state. MySQL still guards durable effects. |
| Popular mentor ranking | Return a precomputed, ordered display list. | SORTED SET; scores come from authoritative metrics, refreshed periodically. |
| Recent activity feed | Serve a bounded temporary feed without making it history storage. | LIST or STREAM; trim length and expire. MySQL retains authoritative history. |

## 2.1 Do not cache these as authoritative

| DATA / DECISION | WHY IT DOES NOT BELONG IN REDIS AS AUTHORITY |
|---|---|
| Identity, roles and account status | MySQL stores the canonical account state. Redis may only support temporary challenge/session state. |
| Mentorship request and allocation status | These are durable lifecycle records, subject to unique constraints and transactional transitions. |
| Availability and confirmed bookings | Concurrent booking must be protected by InnoDB locks/constraints; cache can only show a preview. |
| Session outcomes, feedback, reviews and ratings | These are durable records and reporting inputs. A ranking cache may be derived from them, never replace them. |
| Audit, notification and outbox history | These records need durable traceability and retry state in MySQL. |

**CACHE DESIGN QUESTION**

Before adding any key, answer: what authoritative MySQL query does this key avoid, how is its payload rebuilt, how does it become stale, and what happens if Redis is empty or unavailable?

# 03 / Key naming convention

Use one predictable namespace for all keys. This prevents collisions between environments and makes keys searchable during development. Key names should communicate product, deployment environment, schema version, feature area and entity identity. Avoid names that include email addresses, phone numbers, names or other personal data.

**RECOMMENDED KEY GRAMMAR**

**`ac:{environment}:v{schema}:{feature}:{resource}:{identifier}:{variant}`**
Example: `ac:prod:v1:mentor:profile:482:public`

## 3.1 Naming components

| COMPONENT | RULE | EXAMPLE |
|---|---|---|
| Product prefix | Constant prefix for every AlumniConnect key. | ac |
| Environment | Use dev, test, staging or prod. Never share cache namespaces across environments. | prod |
| Schema version | Increment when the key meaning or payload shape changes incompatibly. | v1 |
| Feature area | Use a stable bounded vocabulary. | mentor, catalog, rec, auth, rate, idem, ranking, feed |
| Resource | Describe the cached object or state. | profile, skills, domains, result, challenge, counter |
| Identifier | Use immutable MySQL IDs, not mutable names. | 482, 901, user_id |
| Variant / dimensions | Add public/private, date, domain, filter digest or other query dimensions as needed. | public, 2026-11-02, f:9e31 |

## 3.2 Rules for stable and safe keys

- Use lowercase ASCII and colon separators. Avoid spaces and arbitrary user-supplied strings.
- Use numeric or opaque stable IDs. A mentor changing their display name must not change their key identity.
- Include all result-shaping filters in recommendation keys. Canonicalise the filter object and hash it (for example, SHA-256 truncated to 12-16 hex characters).
- Include an explicit payload/schema version. Do not silently reuse a key after changing the cached JSON contract.
- Do not encode access tokens, emails, phone numbers, request objectives or free-text biography content in key names.
- Keep environment separation in the key even when different Redis instances are used; it protects local development and test tooling.

## 3.3 Good and bad examples

| GOOD | BAD | REASON |
|---|---|---|
| ac:prod:v1:mentor:profile:482:public | mentor:Sandeep@example.com | No PII; environment/version explicit; stable ID. |
| ac:prod:v1:rec:student:109:filters:8ab31d7e | recommendations:machine-learning:all | Stable ID and exact filter digest avoid collisions. |
| ac:prod:v1:rate:user:109:mentor-search:20261102T1200 | rate:109 | Scope and time bucket are explicit. |

# 04 / Key catalogue - read caches

The following key register is the proposed first-release contract. Field names in payload examples are illustrative public display fields; align them with the final API response model.

| KEY PATTERN | TYPE | PAYLOAD / MEMBER MODEL | TTL |
|---|---|---|---|
| ac:{env}:v1:mentor:profile:{mentor_id}:public | HASH | name, headline, current_title, company_name, experience_years, skills_json, domains_json, profile_version | 600 s |
| ac:{env}:v1:catalog:skills | HASH | field = skill_id; value = compact JSON {name, category, active} | 3600 s |
| ac:{env}:v1:catalog:skill-ids | SET | Members = active skill IDs; membership/filter helper. | 3600 s |
| ac:{env}:v1:catalog:domains | HASH | field = domain_id; value = compact JSON {name, description, active} | 3600 s |
| ac:{env}:v1:catalog:domain-ids | SET | Members = active domain IDs. | 3600 s |
| ac:{env}:v1:rec:student:{student_id}:filters:{digest}:g{epoch} | STRING | JSON candidate list, ranking evidence, generation time and payload version. Exclude private fields. | 300 s |
| ac:{env}:v1:ranking:domain:{domain_id} | SORTED SET | Member = mentor_id; score = display ranking metric. Metrics derived from MySQL. | 300 s |

## 4.1 Mentor profile HASH payload

```
HSET ac:prod:v1:mentor:profile:482:public \
mentor_id 482 \
display_name "Tejas N" \
headline "ML Engineer" \
current_title "Machine Learning Engineer" \
company_name "Stackforce" \
experience_years 5 \
skills_json "[\"Python\",\"MLOps\",\"Docker\"]" \
domains_json "[\"Machine Learning\"]" \
profile_version 17
EXPIRE ac:prod:v1:mentor:profile:482:public 600
```

Do not put private email, phone, internal notes, admin-only fields, authentication data or confidential session notes into the public mentor cache. The PRD requires public directory responses to exclude private account details.

## 4.2 Recommendation result payload

```
{
"schema_version": 1,
"generated_at": "2026-11-02T12:00:00Z",
"student_id": 109,
"candidates": [
{"mentor_id": 482, "score": 0.91,
"reasons": ["3 shared skills", "Machine Learning domain",
"5 years relevant experience"]}
]
}
```

The score and explanation are for ranking and display. Before a response is returned or a request can be submitted, the backend must re-check current account status, visibility, capacity and other eligibility conditions in MySQL. A cached recommendation is never an allocation.

# 05 / Key catalogue - temporary and operational state

| KEY PATTERN | TYPE | PURPOSE / VALUE | TTL / LIMIT |
|---|---|---|---|
| ac:{env}:v1:auth:challenge:{challenge_id} | STRING | One-time challenge state; store a hashed challenge or opaque state, not a password. | 300 s |
| ac:{env}:v1:auth:session:{session_id} | HASH or STRING | Optional short-lived server-side session metadata / revocation marker. Must align with the auth design. | Policy-bound; e.g. 1800 s idle |
| ac:{env}:v1:rate:{subject}:{id}:{route}:{window} | STRING | Integer request counter for a fixed time window. | Window + small buffer |
| ac:{env}:v1:idem:{actor_id}:{operation}:{idempotency_key} | STRING | Request fingerprint + IN_PROGRESS / COMPLETED state + compact response reference. | 86400 s |
| ac:{env}:v1:availability:mentor:{mentor_id}:{date} | STRING or HASH | Optional calendar display preview only; never booking authority. | 15 s |
| ac:{env}:v1:feed:user:{user_id} | LIST or STREAM | Recent display activity; bounded length / retention. | 604800 s |
| ac:{env}:v1:version:mentor:{mentor_id} | STRING | Version counter used to namespace or invalidate mentor-related cache keys. | No TTL |
| ac:{env}:v1:version:recommendations | STRING | Global recommendation cache epoch. Increment after relevant graph / eligibility changes. | No TTL |
| ac:{env}:v1:version:catalog:skills | STRING | Catalogue version used to refresh skill-dependent cache keys. | No TTL |
| ac:{env}:v1:version:catalog:domains | STRING | Catalogue version used to refresh domain-dependent cache keys. | No TTL |

## 5.1 Optional version keys

Version keys are invalidation metadata, not cached business data. Initialise a missing version to 1 or 0 and increment it after the committed business change has been observed. Do not apply a short TTL to version counters: accidental expiry could make an old versioned cache key appear current again. If the Redis instance is rebuilt, increment a deployment-wide cache epoch or change the schema namespace (for example, v2).

## 5.2 Optional availability preview

**STRICT PREVIEW RULE**

An availability preview may be cached for a very short display interval. The booking service must ignore that cached value, read and lock the relevant MySQL availability row, validate overlapping appointments and capacity, then commit the booking and outbox event atomically.

## 5.3 Data minimisation

Prefer a small display projection over serialising entire ORM entities. Cache only what the receiving screen needs. Do not cache user credentials, plaintext OTP values, private contact fields for public access, or complete review/session text. Apply backend authorization even when the data came from a cache.

# 06 / Data structure selection

| STRUCTURE | ALUMNICONNECT USE | WHY THIS STRUCTURE | COMMON COMMANDS |
|---|---|---|---|
| STRING | Recommendation JSON; challenge state; rate counter; idempotency record; version number. | One value per key; simple payload and atomic counter patterns. | GET, SET, SET EX, SET NX, INCR, EXPIRE, TTL |
| HASH | Mentor card fields; skill/domain lookup maps; optional session metadata. | Field-level reads/writes; avoids fetching unrelated fields for compact objects. | HSET, HGET, HMGET, HGETALL, HDEL |
| SET | Active skill/domain ID membership; optional tag/reverse indexes. | Unique membership and fast membership tests; order not guaranteed. | SADD, SREM, SISMEMBER, SMEMBERS, SCARD |
| SORTED SET | Popular mentors / precomputed domain rankings. | Unique member plus numeric score and ordered range retrieval. | ZADD, ZINCRBY, ZREVRANGE / ZRANGE, ZREM |
| LIST | Small recent activity feed when newest-first append/pop is sufficient. | Simple ordered sequence; trim to a bounded number. | LPUSH, LRANGE, LTRIM, RPOP |
| STREAM | Recent activity events where IDs, consumer groups or replay within a short window are useful. | Structured append-only event entries with bounded trimming. | XADD, XREAD, XTRIM, XGROUP |

## 6.1 Selection rules

- **HASH:** use for a compact mentor profile card or a catalogue mapping. Keep fields small and stable.
- **STRING:** use for a JSON blob when the application reads the whole recommendation response as one unit; include a schema version.
- **SET:** use when uniqueness and membership matter, not when order matters.
- **SORTED SET:** use for precomputed ranking. A ranking score is not a booking permission or live eligibility fact.
- **LIST:** use for a lightweight feed. Trim it on each append and never use it as permanent event history.
- **STREAM:** use only if the team needs event-like consumption or consumer-group features; it is optional for the core release.

## 6.2 Avoid premature structure complexity

Do not create a Redis key for every MySQL table. Choose a structure because it supports a real read or temporary-state pattern. For the first implementation, the minimum useful set is: HASH for mentor profiles and reference catalogues; STRING for recommendation JSON, counters, challenges and idempotency; SORTED SET for rankings; LIST for recent activity. Add STREAM only when there is a clear requirement.

# 07 / TTL policy and expiry strategy

The supplied documents require TTL and explicit invalidation but do not prescribe exact numeric expiry values. The durations below are **recommended starting defaults** for the academic project. Measure freshness and hit rate during testing, then adjust them to the final environment and product policy.

| KEY / DATA | STARTING TTL | WHY / POLICY | IMPORTANT CAVEAT |
|---|---|---|---|
| Public mentor profile HASH | 10 min (600 s) | Frequently viewed, relatively stable display data. | Explicitly invalidate on profile, skills, domains, company, experience, visibility or account-status change. |
| Skill / domain catalogue | 60 min (3600 s) | Reference data usually changes less frequently. | Admin catalogue change must invalidate or bump catalogue version. |
| Recommendation result | 5 min (300 s) | Graph and eligibility result may become stale after skill, status, capacity or rating changes. | Recheck current authorization and eligibility in MySQL before return/action. |
| Popular mentor ZSET | 5 min (300 s) | A display ranking can be refreshed periodically. | Never use stale score as a booking or capacity decision. |
| Availability preview | 15 s | Display calendar freshness should be short. | Not used by booking logic; MySQL lock/read is mandatory. |
| Login challenge | 5 min (300 s) | Short-lived verification challenge; expiry is mandatory. | Single-use; attempt count and comparison should be enforced server-side. |
| Rate-limit counter | Window + 5 s | Counter expires just after its fixed-window interval. | Use atomic increment+expiry; define fail-open/fail-closed behaviour. |
| Idempotency result | 24 h (86400 s) | Covers common client retry intervals. | MySQL uniqueness/idempotency protection is still required for durable effects. |
| Recent activity feed | 7 days (604800 s) | Temporary activity display only. | Also cap count/stream length; MySQL retains history. |
| Version counters | No TTL | Invalidation generation metadata. | Reset only by deliberate namespace/cache-epoch migration. |
| Optional auth session metadata | 30 min idle example | Only if server-side session storage is selected. | Must align with access-token/session policy; not specified by source baseline. |

## 7.1 TTL implementation practices

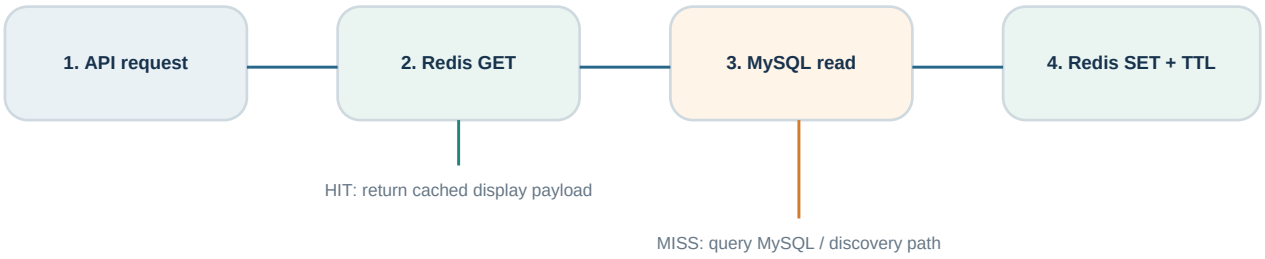
- Set the expiry at the time the value is written: `SET key value EX 600` or write fields then `EXPIRE key 600`.
- Check TTL during debugging with `TTL key`. Redis reports -2 when the key is absent and -1 when it exists without expiry.
- Add small random jitter to high-volume cache TTLs (for example, 600 seconds plus or minus 60 seconds) to avoid many keys expiring simultaneously.
- Do not rely on expiry alone for user-visible freshness. Events must invalidate profile/recommendation/catalogue caches after SQL commit.
- For lists or streams, limit both time and size. TTL alone does not prevent a single key from growing without bound before expiry.

**POLICY STATUS**

All concrete TTLs in this section are proposed defaults, not explicit requirements from the uploaded blueprint or PRD. Record final values in application configuration so they can be tuned without code changes.

# 08 / Cache-aside read path

Cache-aside is the default read pattern for mentor cards, reference catalogues and recent recommendations. The application, not Redis, decides how to rebuild a missing value.




**Authorization still runs; cache never confirms a booking.**

*Figure 2. Cache-aside flow. Authorization executes for both hits and misses.*

## 8.1 Read algorithm

| STEP | ACTION | RESULT |
|---|---|---|
| 1. Authorize | Authenticate the actor and check role/resource permissions before exposing a cached payload. | Unauthorized request is rejected even when Redis contains the key. |
| 2. Build key | Build the environment/version/resource key including IDs, variant and relevant filter digest. | Deterministic cache key. |
| 3. GET | Read Redis. If found, validate payload/schema version and return permitted display fields. | Cache hit. |
| 4. MISS | Read authoritative data from MySQL or invoke the appropriate Neo4j discovery + MySQL eligibility pipeline. | Current source data. |
| 5. SET + TTL | Serialize only the public/safe response fields, write it with TTL and a version marker. | Future cache hit. |
| 6. Respond | Return the response. For any write or booking action, use the normal MySQL transaction path. | Cached read does not grant write authority. |

## 8.2 Cache stampede protection

When many requests miss the same hot key, they can all query MySQL. For expensive mentor-card or recommendation rebuilds, use a short-lived lock key with `SET lock-key token NX EX 10`. The lock owner rebuilds the cache; other requests briefly wait, serve a safe stale display copy if one exists, or perform a bounded fallback. Always release a lock only if the stored token matches the owner token (a Lua compare-and-delete script); never blindly DEL another worker's lock.

## 8.3 Cache payload validation

- Include `schema_version` and a generated timestamp in JSON response caches.
- Reject malformed or unsupported cached data; delete it and rebuild from the authoritative source.
- Do not cache a response that includes fields the current actor is not allowed to view.
- On a cold cache or Redis outage, fall back to the normal MySQL/discovery path where safe; apply timeouts so Redis does not stall the request.

# 09 / Cache invalidation design

TTL limits how long a value can survive, but it is not a complete freshness strategy. AlumniConnect should invalidate relevant cache entries after the authoritative MySQL transaction commits. The supplied requirements explicitly call for profile-change invalidation and outbox-driven cache invalidation.

## 9.1 Invalidation mechanisms

| MECHANISM | HOW IT WORKS | WHEN TO USE |
|---|---|---|
| Targeted DEL | Delete a known profile, catalogue or availability-preview key. | A single entity key has a known deterministic name. |
| Version / epoch bump | Increment a version key used in derived cache keys; new reads naturally use a new namespace. Old values expire by TTL. | Recommendation results or caches that depend on many entities. |
| Reverse index set (advanced) | Maintain a SET of cache keys associated with a mentor/student/domain and delete those members on change. | Only if the project later needs highly targeted invalidation at larger scale; keep index cleanup bounded. |
| TTL expiry | Allow an entry to disappear after a maximum age. | Safety net for every ordinary cache; not a substitute for event invalidation. |

## 9.2 Recommended first-release strategy

**Mentor profile:** targeted delete of the mentor public-card HASH, then increment the mentor version. **Recommendation results:** increment one global recommendation epoch after any relevant mentor/student skill/domain, mentor visibility/status, capacity/allocation, or rating change. Recommendation keys include the epoch. This is intentionally simple and safe for a small academic dataset; it invalidates more results than strictly necessary, but old results expire naturally.

```
INCR ac:prod:v1:version:mentor:482
DEL ac:prod:v1:mentor:profile:482:public
INCR ac:prod:v1:version:recommendations
```

For a catalogue change, delete or version-bump the appropriate catalogue key and increment the recommendation epoch. For an availability edit or booking/cancellation, delete any display-preview key for the affected mentor/date; the confirmed booking remains in MySQL.

## 9.3 Never use production-wide wildcard deletion per entity

Avoid KEYS `ac:prod:*` in application code. It can block Redis while scanning a large keyspace. Use deterministic keys, version namespaces and direct deletion. Use SCAN only for controlled maintenance or debugging, not as the normal invalidation mechanism.

# 10 / Event-to-cache invalidation matrix

| COMMITTED MYSQL EVENT | REDIS ACTION | RECOMMENDATION EFFECT |
|---|---|---|
| MentorProfileUpdated | DEL mentor public profile key; INCR mentor version; INCR recommendation epoch. | Refresh mentor card and candidate results. |
| MentorSkillsUpdated | Same as above. | Shared-skill paths and mentor rankings may change. |
| MentorDomainsUpdated | Same as above. | Domain match reasons and rankings may change. |
| MentorCompanyOrExperienceUpdated | Delete profile; bump mentor and recommendation versions. | Career relevance and experience match may change. |
| MentorVisibilityOrAccountStatusChanged | Delete profile; bump mentor and recommendation versions. | Mentor may need to disappear from public discovery. |
| StudentSkillsUpdated / StudentInterestsUpdated | Bump student version if per-student keys are used; bump recommendation epoch. | The student's recommendation set changes. |
| SkillCatalogueChanged / DomainCatalogueChanged | DEL catalogue key (or bump catalogue version); bump recommendation epoch. | Search filters and match labels may change. |
| MentorAllocationCreated / AllocationEnded | Bump recommendation epoch or mentor eligibility version. | Capacity/eligibility may change; MySQL rechecks it regardless. |
| MentorAvailabilityChanged | DEL availability preview for mentor/date. | Refresh calendar display only. |
| BookingConfirmed / BookingCancelled | DEL affected availability preview; optionally bump mentor eligibility/ranking epoch. | Never use cache to confirm a slot. |
| MentorReviewCreated / FeedbackAggregatesUpdated | DEL or refresh ranking ZSET; bump recommendation epoch if quality affects ranking. | Rebuild ranking from current MySQL aggregates. |
| RecentActivityCreated | LPUSH/XADD feed entry; trim and refresh TTL. | Temporary display activity only. |

## 10.1 Minimal invalidation event payload

```
{
"event_id": "outbox-uuid",
"event_type": "MentorSkillsUpdated",
"aggregate_type": "Mentor",
"aggregate_id": 482,
"aggregate_version": 17,
"occurred_at": "2026-11-02T12:00:00Z"
}
```

The event should carry stable IDs and enough context for deterministic invalidation. Avoid putting full private profile records in the outbox message when a resource ID and version are sufficient. Worker retries must be idempotent.

# 11 / Transactional outbox integration

Cache invalidation must not occur before the MySQL commit. Otherwise, a transaction that later rolls back could cause a cache miss and rebuild from unchanged rows, or the inverse: a cache might be repopulated with data that was not committed. The application architecture therefore records the invalidation event in the same transaction as the business update.

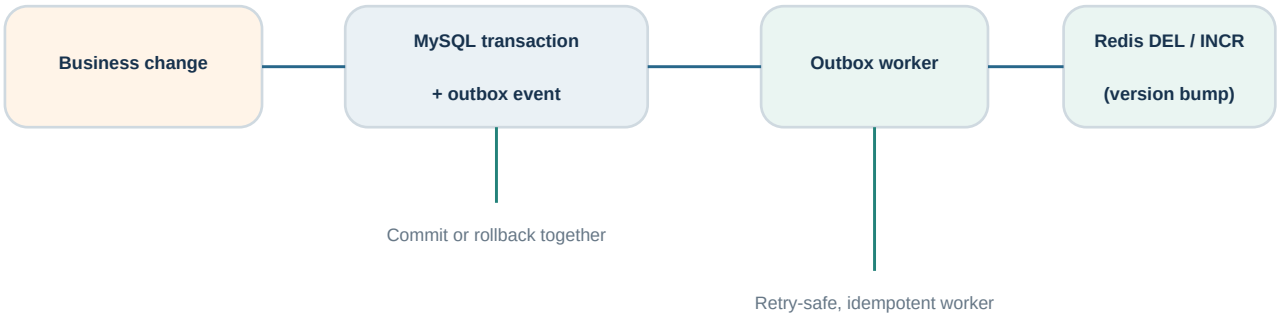



*Figure 3. SQL update and outbox row commit together; Redis invalidation is a retryable side effect.*

## 11.1 Required sequence

| STAGE | OPERATION | CONSISTENCY PROPERTY |
|---|---|---|
| 1. Validate | FastAPI authorizes actor and validates profile update. | No secondary store is mutated yet. |
| 2. SQL transaction | Update MySQL profile/skill/domain rows and insert outbox event in the same transaction. | Both commit or both roll back. |
| 3. Commit | MySQL transaction commits. | The new business data is authoritative. |
| 4. Worker | Worker reads committed outbox row and applies cache invalidation. | Retry possible if Redis was unavailable. |
| 5. Mark complete | Worker records processed status/attempt metadata in MySQL. | Operationally observable delivery. |

## 11.2 Worker reliability requirements

- Process only committed outbox rows.
- Use event IDs and idempotent Redis commands: DEL of a missing key is safe; repeated INCR is not idempotent unless deduplicated.
- A repeated invalidation may cause an extra cache miss but cannot duplicate a business write. If strict event deduplication is needed, use one Lua script (or equivalent atomic Redis operation) that checks the event marker, applies DEL/version changes and records the marker as one atomic unit. Do not write a marker in a separate command before invalidating: a worker crash between those steps could suppress a required retry.
- Track pending, processed and failed status, attempts, last error and next retry time in MySQL.
- Retry transient errors with bounded backoff. Do not silently discard failures.
- If Redis is unavailable, do not roll back a committed profile update. Leave the outbox event pending and retry.

**IMPORTANT RETRY DETAIL**

DEL can safely be replayed. INCR can over-invalidate if the same outbox event is processed twice. Use event-level deduplication or deterministic version values so a retry does not repeatedly advance the cache epoch.

# 12 / Temporary state: challenges, rate limits and idempotency

These keys are operational helpers. They are not durable business records. Keep them narrowly scoped, short-lived and protected from exposure in logs.

## 12.1 Login challenge

```
SET ac:prod:v1:auth:challenge:ch_7f2a <hashed-or-opaque-state> NX EX 300
GET ac:prod:v1:auth:challenge:ch_7f2a
DEL ac:prod:v1:auth:challenge:ch_7f2a
```

Recommended behaviour: generate a high-entropy challenge ID; store a hash/opaque verifier rather than the raw password or reusable credential; expire in five minutes; limit verification attempts; consume it once after successful validation. Exact login method and retry policy are open application-security decisions.

## 12.2 Fixed-window rate limiting

Example key: `ac:prod:v1:rate:user:109:mentor-search:20261102T1200`. Increment the counter and set the expiry atomically on the first increment. A simple two-command INCR followed by EXPIRE can race if the process fails between commands; use a Lua script or a transaction/pipeline pattern that guarantees first-write expiry.

```
-- Lua: increment and apply expiry once, atomically
local n = redis.call('INCR', KEYS[1])
if n == 1 then
redis.call('EXPIRE', KEYS[1], ARGV[1])
end
return n

-- Example: fixed 60-second bucket; enforce threshold in FastAPI
```

Choose limits by operation risk: mentor search may have a higher limit than login verification or request submission. The numeric limits are policy decisions and are intentionally not hard-coded in this document. Rate limiting should not be treated as a replacement for authentication, authorization or abuse monitoring.

## 12.3 Request idempotency

```
SET ac:prod:v1:idem:109:booking:create:idem_abc \
"<request-hash>|IN_PROGRESS" NX EX 86400
```

When the request completes, store a compact outcome or stable result reference and status COMPLETED. If a retry arrives with the same key and same request fingerprint, return the stored result. If the same key is reused for a different request body, reject it. The booking itself remains protected by MySQL row locks, unique constraints and a transaction; Redis idempotency alone is not enough.

## 12.4 Failure policy

| KEY CATEGORY | IF REDIS IS UNAVAILABLE / KEY IS EVICTED |
|---|---|
| Profile/catalogue cache | Fall back to MySQL; slower but correct. |
| Recommendation cache | Recompute via Neo4j + MySQL validation, or return a controlled retry if dependencies are unavailable. |
| Login challenge | Fail the verification attempt safely and allow a new challenge; do not bypass verification. |
| Rate limit | Use application/API fallback or return controlled retry; never treat a missing counter as identity proof. |
| Idempotency | Rely on MySQL unique/idempotency constraints for durable writes; do not allow duplicate booking effects. |

# 13 / Redis command cookbook

These examples are intended for Redis CLI inspection during development. Replace IDs and environment names with test data. Avoid running destructive commands against production.

## 13.1 Mentor cache operations

```
HSET ac:dev:v1:mentor:profile:482:public display_name "Tejas N" headline "ML Engineer" experience_years 5
EXPIRE ac:dev:v1:mentor:profile:482:public 600
HGETALL ac:dev:v1:mentor:profile:482:public
TTL ac:dev:v1:mentor:profile:482:public
DEL ac:dev:v1:mentor:profile:482:public
```

## 13.2 Skill/domain catalogue

```
HSET ac:dev:v1:catalog:skills 17 "{\"name\":\"Python\",\"active\":true}"
SADD ac:dev:v1:catalog:skill-ids 17 18 19
SISMEMBER ac:dev:v1:catalog:skill-ids 17
EXPIRE ac:dev:v1:catalog:skills 3600
EXPIRE ac:dev:v1:catalog:skill-ids 3600
```

## 13.3 Recommendation JSON cache

```
SET ac:dev:v1:rec:student:109:filters:8ab31d7e:g12 \
'{"schema_version":1,"candidates":[{"mentor_id":482,"score":0.91}]}' EX 300
GET ac:dev:v1:rec:student:109:filters:8ab31d7e:g12
```

## 13.4 Popular mentor ranking

```
ZADD ac:dev:v1:ranking:domain:4 91.0 482 84.0 517 78.5 614
ZREVRANGE ac:dev:v1:ranking:domain:4 0 9 WITHSCORES
EXPIRE ac:dev:v1:ranking:domain:4 300
```

## 13.5 Recent feed

```
LPUSH ac:dev:v1:feed:user:109 \
'{"type":"BOOKING_CONFIRMED","booking_id":9001,"at":"2026-11-02T12:00:00Z"}'
LTRIM ac:dev:v1:feed:user:109 0 49
EXPIRE ac:dev:v1:feed:user:109 604800
LRANGE ac:dev:v1:feed:user:109 0 9
```

## 13.6 Invalidation and TTL inspection

```
DEL ac:dev:v1:mentor:profile:482:public
INCR ac:dev:v1:version:mentor:482
INCR ac:dev:v1:version:recommendations
TTL ac:dev:v1:rec:student:109:filters:8ab31d7e:g12
SCAN 0 MATCH ac:dev:v1:mentor:* COUNT 100
```

**DEBUGGING CAUTION**

Use SCAN for inspection/maintenance, not KEYS over a live large keyspace. The example above shows key discovery in a development namespace only.

# 14 / FastAPI integration templates

The following Python examples use redis-py style APIs. They are implementation templates: add typed response models, dependency injection, logging redaction, connection timeouts and exception handling to match the final FastAPI codebase.

## 14.1 Cache-aside mentor card

```
import json
from redis.asyncio import Redis

PROFILE_TTL_SECONDS = 600

async def get_public_mentor_card(
redis: Redis, mysql_repo, mentor_id: int, actor
) -> dict:
# Authorization is checked before serving cached display fields.
await mysql_repo.authorize_public_mentor_discovery(actor)

key = f"ac:prod:v1:mentor:profile:{mentor_id}:public"
cached = await redis.hgetall(key)
if cached:
return decode_public_card(cached)

# Rebuild only from the public-safe MySQL projection.
card = await mysql_repo.get_public_mentor_card(mentor_id)
if card is None:
raise NotFoundError("Mentor not found")

# Never put private email/phone/admin fields in this payload.
payload = encode_public_card(card)
async with redis.pipeline(transaction=True) as pipe:
pipe.hset(key, mapping=payload)
pipe.expire(key, PROFILE_TTL_SECONDS)
await pipe.execute()
return card
```

## 14.2 Post-commit invalidation

```
async def update_mentor_skills(mysql, outbox_repo, mentor_id, skill_ids):
async with mysql.transaction() as tx:
await tx.mentor_skills.replace(mentor_id, skill_ids)
await outbox_repo.insert(
tx,
event_type="MentorSkillsUpdated",
aggregate_type="Mentor",
aggregate_id=mentor_id,
)
# SQL commit has completed here. A best-effort direct invalidation can
# reduce staleness; the outbox worker remains the retryable mechanism.
# If direct invalidation fails, the committed outbox event is still pending.
```

## 14.3 Cache epoch in a recommendation key

```
async def recommendation_key(redis, student_id, filters_digest):
epoch_key = "ac:prod:v1:version:recommendations"
epoch = await redis.get(epoch_key)
epoch = epoch or "0"
return (
f"ac:prod:v1:rec:student:{student_id}:"
f"filters:{filters_digest}:g{epoch}"
)
```

For very high-volume reads, fetch the version and recommendation entry with MGET/pipelining or a Lua script. The first-release project can use a straightforward implementation until profiling proves it is a bottleneck.

# 15 / Operations, memory and failure handling

Redis is a fast in-memory system, but memory is finite. Cache values should be bounded, observable and disposable. The project should choose its max-memory policy based on which data is stored.

## 15.1 Eviction policy considerations

| POLICY CONSIDERATION | ALUMNICONNECT GUIDANCE |
|---|---|
| General cache eviction | For a cache-dominant instance, a least-recently-used or least-frequently-used all-key eviction policy can be considered after verifying every key can safely be recreated. |
| Version counters | Do not allow unplanned eviction/expiry to reset cache generations while old keys could still be read. Use a persistent version key or change the deployment cache epoch after rebuild. |
| Auth/rate/idempotency keys | Eviction can invalidate challenges early, reset rate counters or remove idempotency results. Treat these as short-lived helpers, not the only security or business guard. MySQL must protect durable writes. |
| Memory pressure | Monitor used memory, evictions, rejected writes, key cardinality and large payloads. Keep feed/list values bounded. |
| Persistence | Because cache values are rebuildable, persistence is not required for their correctness. Temporary security state and retry safety must have an explicit failure policy if Redis restarts. |

## 15.2 Recommended operational controls

- Set connection and command timeouts in FastAPI; Redis latency must not block requests indefinitely.
- Use a connection pool and a separate Redis client configuration for production and tests.
- Apply authentication, network isolation and TLS where the deployment supports it; never expose Redis publicly.
- Use a dedicated Redis ACL user with only required key prefixes and commands when supported by the deployment.
- Separate cache keys from temporary auth/operational keys by prefix. For production scale, consider separate Redis instances/policies if eviction behaviour must differ.
- Do not log verification secrets, idempotency response bodies or full cached personal data.

## 15.3 Graceful degradation

If Redis is unavailable, the application should still behave correctly for ordinary reads by falling back to MySQL or the normal discovery pipeline. It may be slower. For temporary security operations, fail safely rather than skipping verification. For idempotent writes, rely on the relational constraints and transaction design so a Redis restart cannot create duplicate bookings or allocations.

# 16 / Privacy and security rules for cached data

| CONTROL | IMPLEMENTATION RULE |
|---|---|
| Public profile projection | Cache only the fields intentionally exposed in the mentor directory. Build this from a least-privilege SQL view/repository projection. |
| Role-aware authorization | Authenticate and authorize every request before serving a cache hit. Cache keys are not access-control tokens. |
| Private records | Do not put private contact details, internal notes, administrative fields or complete session notes in public caches. |
| Temporary challenges | Use random opaque IDs and hashed/opaque challenge values; expire them; enforce one-time consumption and attempt limits. |
| Idempotency data | Store a request fingerprint and compact response reference where possible. Do not include sensitive request bodies in key names. |
| Key isolation | Prefix by environment and application version; use separate credentials/ACLs for deployment environments. |
| Data lifecycle | Keep TTLs short for temporary state and bounded for feeds. Delete relevant cache keys after deactivation/visibility changes; historical business records remain in MySQL. |
| Auditability | Redis is not the audit log. Authoritative administrative and business history remains in MySQL. |

## 16.1 Public profile example

**Safe fields (illustrative):** mentor ID, display name, headline, professional title, company display name, years of experience, public skills, public domain expertise, profile completeness flag.

**Do not include:** login email, phone number, raw account status details, admin moderation notes, private internal biography, access tokens, password hashes or participant-only session notes. Exact visibility is controlled by the approved product privacy model.

| SECURITY RULE |
|---|
| A cached object is still data. Its presence in Redis does not make it public, and a cached result cannot bypass the role/ownership checks defined by the API and business rules. |

# 17 / Monitoring and performance measurements

Measure Redis as part of the whole API path. A high cache hit rate is useful only when the payload is correct and the read path actually improves response time. Record baseline and post-cache values against the same test data and environment.

| METRIC | WHAT IT TELLS YOU | HOW TO USE IT |
|---|---|---|
| Cache hit ratio | How often a requested cache entry exists. | Track separately for mentor profiles, catalogues and recommendations. |
| Cache miss ratio | How often the application rebuilds a value. | High misses can indicate short TTLs, key mismatch or invalidation churn. |
| Redis command latency | Time spent in Redis calls. | Watch p50/p95; use timeouts and connection pooling. |
| End-to-end API p50/p95 | Whether caching improves the product API. | Compare before/after for mentor card and discovery endpoints. |
| Evicted keys / memory used | Whether memory pressure is changing cache behaviour. | Investigate oversized payloads, TTLs and key cardinality. |
| Outbox pending age / retry count | Whether invalidation events are being delayed. | Alert on sustained backlog; inspect failed event types. |
| Invalidation-to-refresh time | How long stale display data can remain after a committed update. | Measure with a profile-change test and record the observed delay. |
| Rebuild errors | Whether a miss can be repopulated from source systems. | Log structured errors; do not cache empty error responses for long. |

## 17.1 Useful Redis CLI checks

```
INFO memory
INFO stats
INFO commandstats
DBSIZE
SCAN 0 MATCH ac:prod:v1:mentor:* COUNT 100
TTL ac:prod:v1:mentor:profile:482:public
```

## 17.2 Performance test procedure

- Run the mentor-profile endpoint with an empty Redis cache; record MySQL query count and p50/p95 latency.
- Repeat the same request to produce a cache hit; record Redis calls and API latency.
- Change the mentor profile in MySQL and commit its outbox event; verify the old cache cannot be served afterwards.
- Temporarily stop Redis; confirm that a profile read falls back correctly and booking correctness remains unchanged.
- Run concurrent booking attempts with a stale availability preview in Redis; verify exactly one booking is confirmed by MySQL.

# 18 / Verification and acceptance tests

| TEST ID | SCENARIO | EXPECTED RESULT / EVIDENCE |
|---|---|---|
| RED-01 | First mentor-card request with empty cache. | API reads public-safe data from MySQL, returns card and writes HASH with TTL. |
| RED-02 | Repeat the same card request. | Cache hit returns identical public projection; authorization still executes. |
| RED-03 | Wait for profile TTL to expire. | Key disappears; next request rebuilds the value from MySQL. |
| RED-04 | Update mentor headline/experience. | After committed update and outbox processing, profile key is invalidated and next read is fresh. |
| RED-05 | Update mentor skills/domains. | Profile and recommendation caches are invalidated; graph sync may be asynchronous. |
| RED-06 | Admin updates skill/domain catalogue. | Catalogue cache is deleted or version-bumped; next read loads updated catalogue. |
| RED-07 | Two students request recommendations around a mentor update. | Old recommendation epoch is not reused after invalidation; MySQL still validates current eligibility. |
| RED-08 | Read a cached mentor card as unauthorized actor. | Backend rejects request; Redis hit does not bypass authorization. |
| RED-09 | Inspect public cached payload. | No private email/phone, tokens, passwords or admin-only fields are present. |
| RED-10 | Redis is unavailable during profile read. | Application falls back to MySQL or returns controlled failure; no incorrect business state. |
| RED-11 | Outbox worker retries the same invalidation event. | Duplicate event is safely suppressed; no unbounded epoch increment. |
| RED-12 | Two users book one slot while preview cache says OPEN. | Exactly one booking commits in MySQL; the other receives conflict/unavailable. |
| RED-13 | Rate-limit key increments across one time bucket. | Counter increments atomically and expires after the configured window. |
| RED-14 | Same idempotency key is retried with identical payload. | Same outcome is returned; duplicate effect is not created. |
| RED-15 | Same idempotency key reused with different payload. | Request is rejected as a key/fingerprint conflict. |
| RED-16 | Recent activity is appended repeatedly. | Feed remains within the configured max length and TTL; MySQL history remains complete. |

## 18.1 Minimum demo evidence

- Redis CLI output showing HSET/HGETALL, SET/GET, SADD/SISMEMBER, ZADD/ZREVRANGE, LPUSH/LTRIM and TTL.
- Before/after screenshots or logs showing cache miss, cache hit and TTL expiry.
- A committed MySQL profile change followed by outbox processing and Redis invalidation.
- A negative test proving a cache hit does not allow an unauthorized read or confirm a booking.

# 19 / Implementation roadmap and configuration register

| PHASE | IMPLEMENTATION TASKS | EXIT CRITERION |
|---|---|---|
| 1. Foundation | Create Redis client/pool; define environment namespace and schema version; add health check and configuration. | Backend connects securely; tests use isolated key prefix. |
| 2. Read caches | Implement mentor profile HASH and skill/domain catalogue caches using cache-aside. | Hit, miss, expiry and MySQL fallback tests pass. |
| 3. Recommendation cache | Cache recent recommendation response with a canonical filter digest and global epoch. | Student/mentor/catalogue changes invalidate results; MySQL eligibility check remains. |
| 4. Outbox invalidation | Add event handlers for profile, skill, domain, status, capacity and catalogue changes. | Worker retries are safe; invalidation events are observable. |
| 5. Temporary state | Add login challenge, rate limit and idempotency helpers. | TTL and replay behaviour tested; MySQL guards critical business writes. |
| 6. Ranking/feed | Add ZSET ranking and bounded feed if they support actual screens. | Sorted ranking/feed is derived; history remains MySQL. |
| 7. Performance & demo | Measure endpoint p50/p95, cache hits/misses, memory and invalidation delay. | Evidence documented; TTL defaults tuned. |

## 19.1 Proposed configuration values

```
REDIS_URL=redis://localhost:6379/0
REDIS_KEY_PREFIX=ac:dev:v1
REDIS_CONNECT_TIMEOUT_MS=500
REDIS_SOCKET_TIMEOUT_MS=500
REDIS_TTL_MENTOR_PROFILE_SECONDS=600
REDIS_TTL_CATALOG_SECONDS=3600
REDIS_TTL_RECOMMENDATION_SECONDS=300
REDIS_TTL_AVAILABILITY_PREVIEW_SECONDS=15
REDIS_TTL_LOGIN_CHALLENGE_SECONDS=300
REDIS_TTL_IDEMPOTENCY_SECONDS=86400
REDIS_TTL_ACTIVITY_FEED_SECONDS=604800
```

These are example development configuration values. Use secret management for credentials; do not commit production Redis URLs or passwords. Keep environment-specific configuration outside source control.

## 19.2 Team decisions to confirm

- Final profile-card fields and public/private visibility rules.
- Whether recommendation results are cached before or after MySQL eligibility checks; this specification recommends caching only the response shape that is revalidated at read/action time.
- Actual rate-limit thresholds per endpoint and actor type.
- Login challenge type, maximum attempts, resend cooldown and session/token expiry policy.
- Idempotency retention window and MySQL durable duplicate-prevention mechanism.
- Redis deployment eviction policy and whether critical temporary state merits a separate instance.
- Whether recent activity uses LIST or STREAM; core release can start with LIST.

# 20 / Final design summary

The Redis layer for AlumniConnect should stay intentionally small: a few read caches, a few short-lived operational keys and optional precomputed display views. A compact design is easier to understand, test and explain during the DBMS evaluation.

| AREA | RECOMMENDED DEFAULT | SOURCE-ALIGNED REASON |
|---|---|---|
| Mentor card | HASH; 10-minute TTL; targeted invalidation. | Frequently viewed public profile cache; invalidate after profile change. |
| Skill/domain catalogue | HASH + optional SET; 60-minute TTL. | Reference data lookup; bounded TTL or explicit invalidation. |
| Recommendation cache | STRING JSON; 5-minute TTL; recommendation epoch. | Recent computed results; invalidate when relevant profile/skill data changes. |
| Rate limiting | STRING counter; fixed-window TTL. | Short time-window controls. |
| Login challenge | STRING; 5-minute expiry. | Short-lived verification state; expiry mandatory. |
| Idempotency | STRING fingerprint/result; 24-hour TTL; SQL duplicate protection. | Short-lived client retry protection. |
| Popular ranking | SORTED SET; 5-minute TTL. | Precomputed ranking view; source metrics in MySQL. |
| Activity feed | LIST; 7-day TTL and length cap. | Temporary activity view; permanent history in MySQL. |
| Availability display | Optional short-lived preview; 15-second TTL. | Display convenience only; never booking authority. |

**THE RULE TO REMEMBER**

MySQL decides what is true. Redis decides whether a previously computed display value can be reused for a short time. Once the underlying MySQL data changes, a committed outbox event invalidates or versions the affected cache. When in doubt, rebuild from MySQL.

A practical test of the design: switch Redis off. AlumniConnect may become slower, but it must not accept an invalid request, allocate a duplicate mentor, double-book a slot, expose private data or lose an authoritative session/review record.

# Appendix A / Key register quick reference

| KEY | TYPE | TTL |
|---|---|---|
| ac:{env}:v1:mentor:profile:{mentor_id}:public | HASH | 600 s |
| ac:{env}:v1:catalog:skills | HASH | 3600 s |
| ac:{env}:v1:catalog:skill-ids | SET | 3600 s |
| ac:{env}:v1:catalog:domains | HASH | 3600 s |
| ac:{env}:v1:catalog:domain-ids | SET | 3600 s |
| ac:{env}:v1:rec:student:{student_id}:filters:{digest}:g{epoch} | STRING | 300 s |
| ac:{env}:v1:ranking:domain:{domain_id} | SORTED SET | 300 s |
| ac:{env}:v1:auth:challenge:{challenge_id} | STRING | 300 s |
| ac:{env}:v1:auth:session:{session_id} | HASH / STRING | Policy-bound |
| ac:{env}:v1:rate:{subject}:{id}:{route}:{window} | STRING | Window + buffer |
| ac:{env}:v1:idem:{actor_id}:{operation}:{key} | STRING | 86400 s |
| ac:{env}:v1:availability:mentor:{mentor_id}:{date} | STRING / HASH | 15 s |
| ac:{env}:v1:feed:user:{user_id} | LIST / STREAM | 604800 s + cap |
| ac:{env}:v1:version:mentor:{mentor_id} | STRING | No TTL |
| ac:{env}:v1:version:recommendations | STRING | No TTL |
| ac:{env}:v1:version:catalog:skills | STRING | No TTL |
| ac:{env}:v1:version:catalog:domains | STRING | No TTL |

## Appendix B / Definitions

| TERM | MEANING IN THIS DESIGN |
|---|---|
| Cache-aside | The application checks Redis first; on a miss it reads from the source and caches a rebuildable result. |
| TTL | Time to live; the expiry duration associated with a Redis key. |
| Cache invalidation | Deleting or versioning a cached value after the source data changes. |
| Cache epoch | A version number embedded in derived keys so a new generation is used after a change. |
| Outbox | A durable MySQL event record inserted in the same transaction as the business change. |
| Idempotency key | A client-supplied unique request token used to make retries safe. |
| Cache stampede | A burst of simultaneous misses that causes many requests to rebuild the same value. |

# Appendix C / Source basis and design traceability

This document preserves the responsibilities and limits in the supplied AlumniConnect project materials. Exact TTL values, key strings, Redis scripts and code snippets are implementation recommendations added to make the Redis work concrete; they are not quoted numeric requirements from the source documents.

| SOURCE | RELEVANT CONTENT USED IN THIS SPECIFICATION |
|---|---|
| Alumni–Student Mentorship Platform: DBMS Project Blueprint (34 pp.) | Redis performance layer; mentor-profile and catalogue caches; recommendation cache; temporary login challenge; rate limiting; idempotency; sorted-set ranking; list/stream activity feed; TTL and cache invalidation; Redis must not decide booking availability. |
| AlumniConnect Application Architecture (20 pp.) | Section 7.1, p. 9: Redis workload-to-structure mapping and cache-aside read pattern. Section 8, p. 11: transactional outbox, mentor-skill change sequence, retryability and MySQL precedence. Section 9, pp. 12-13: MySQL booking validation and concurrency. |
| AlumniConnect Product Requirements Document (13 pp.) | FR-20 and FR-23, p. 7: transactional outbox drives projection updates and Redis invalidation; cache-aside, TTL and invalidation; hit/miss/expiry acceptance. Section 6.4, p. 9: MySQL source of truth, Redis rebuildable, explicit invalidation, no booking authority. |
| AlumniConnect Workflow and State Diagrams (25 pp.) | Orientation, pp. 2-3: Redis caches mentor cards, skills/domains and recommendations; short-lived state; booking and allocation remain in MySQL. |
| AlumniConnect Business Rules Catalogue (31 pp.) | Cross-store rules, pp. 26-27: MySQL wins, Redis rebuildable and TTL/invalidation, current eligibility in MySQL, slot checks in SQL. PRO-005: invalidate mentor and recommendation caches after relevant profile changes. |
| AlumniConnect Conceptual ER Model (18 pp.) | Conceptual domain and system-of-record boundary; Redis is a supporting store and not an additional business entity. |

## Design interpretation notes

- The source documents identify the Redis data structures but do not set a mandatory TTL number. The defaults in Section 7 are proposed starting values.
- The source documents require invalidation after authoritative SQL commits. The global recommendation epoch is a simple implementation choice for this project scale; targeted reverse indexes are an optional later optimisation.
- The source documents require short-lived challenge, rate-limit and idempotency support. The exact authentication/session and rate-limit policies remain decisions for the team.
- Optional vector/semantic search is a later phase in the blueprint. It is not a core dependency of this Redis design.

**END OF SPECIFICATION**

Use this document alongside the Relational Design Specification, Business Rules Catalogue, Application Architecture, Neo4j Graph Design and Workflow/State Diagrams. Update the version and key register whenever the implementation contract changes.